

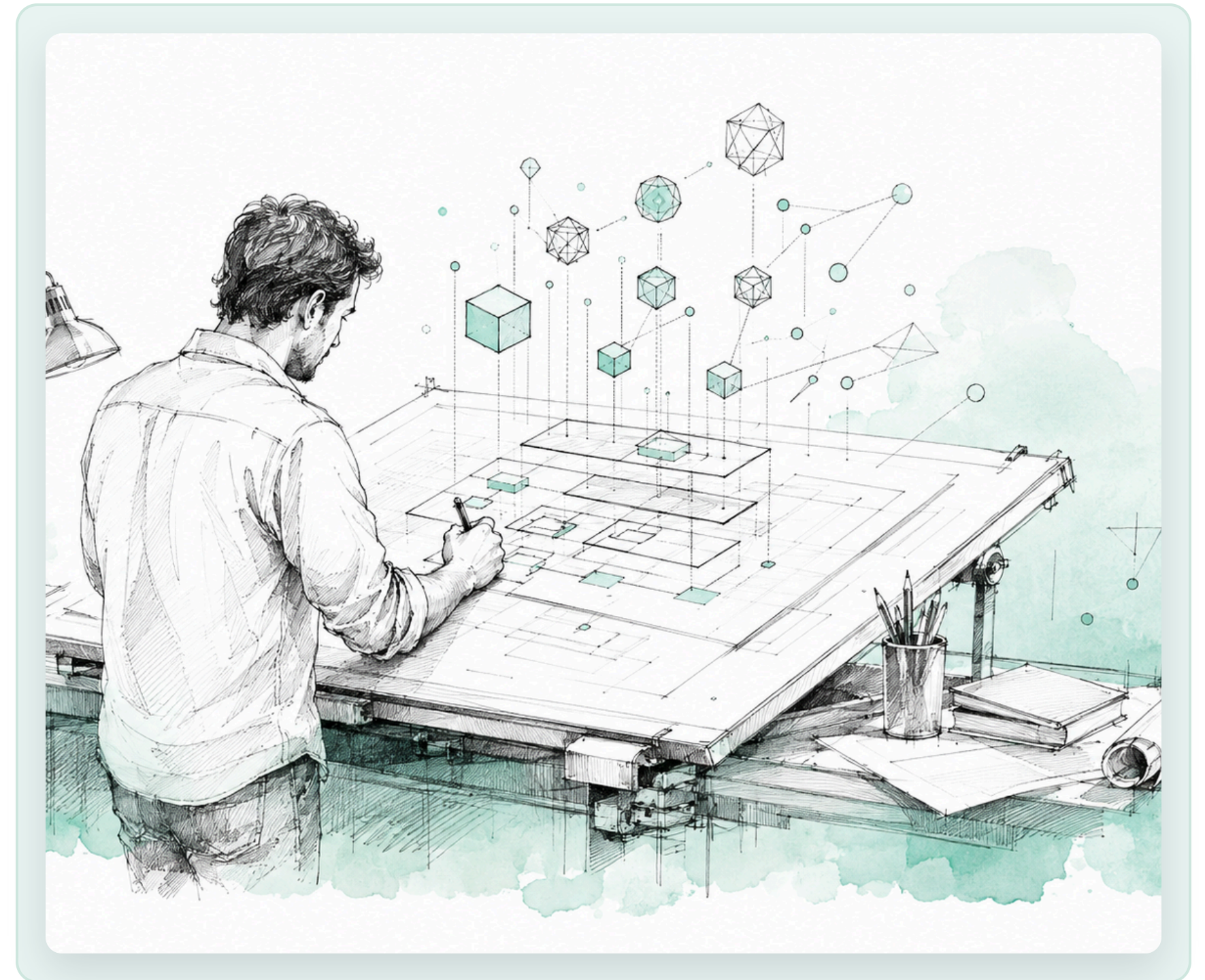
SCENE 01 • INTRODUCTION

If I Were Rebuilding Netflix's Engineering Org With AI

A practical proposal for an AI Native engineering organization.

"How can we give great engineers an engineering system capable of multiplying their capabilities?"

- ▲ This is not a description of Netflix's actual internal architecture — it's a thought experiment using technologies and practices available today.
- ▲ The interesting part is not AI replacing engineers.



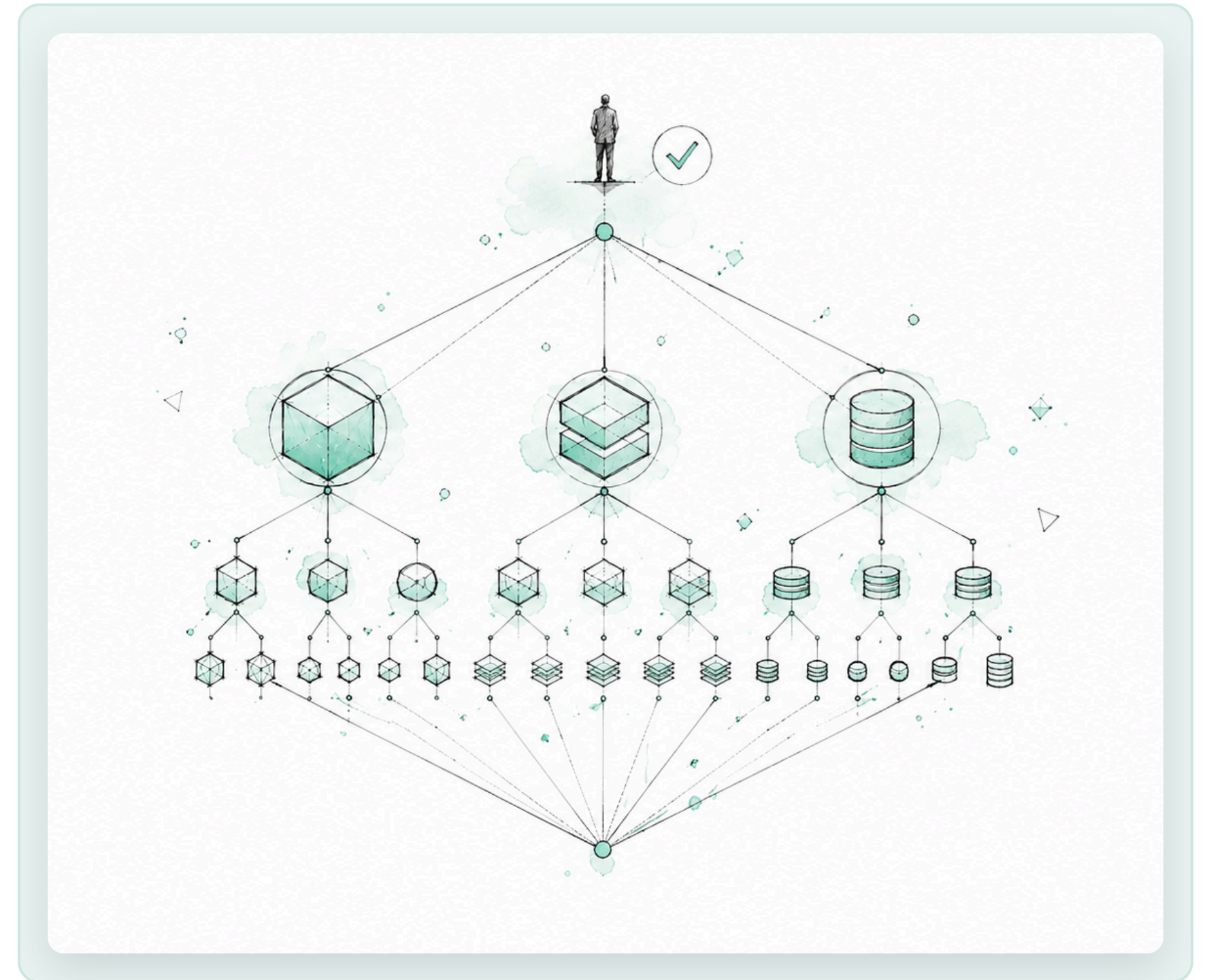
SCENE 02 • HUMAN DECISION

It Starts With a Human Decision

"Improve personalized recommendations for users watching content on mobile devices."

```
HUMAN → Product Intent → Specification  
↓  
AI Orchestrator  
↓  
Product Agent · Architecture Agent · Data Agent  
↓  
Engineering Agents  
↓  
Backend · Mobile · ML Agent  
↓  
Validation → HUMAN REVIEW
```

- ▲ The human defines what and why.
- ▲ The AI system helps determine how and in what sequence.



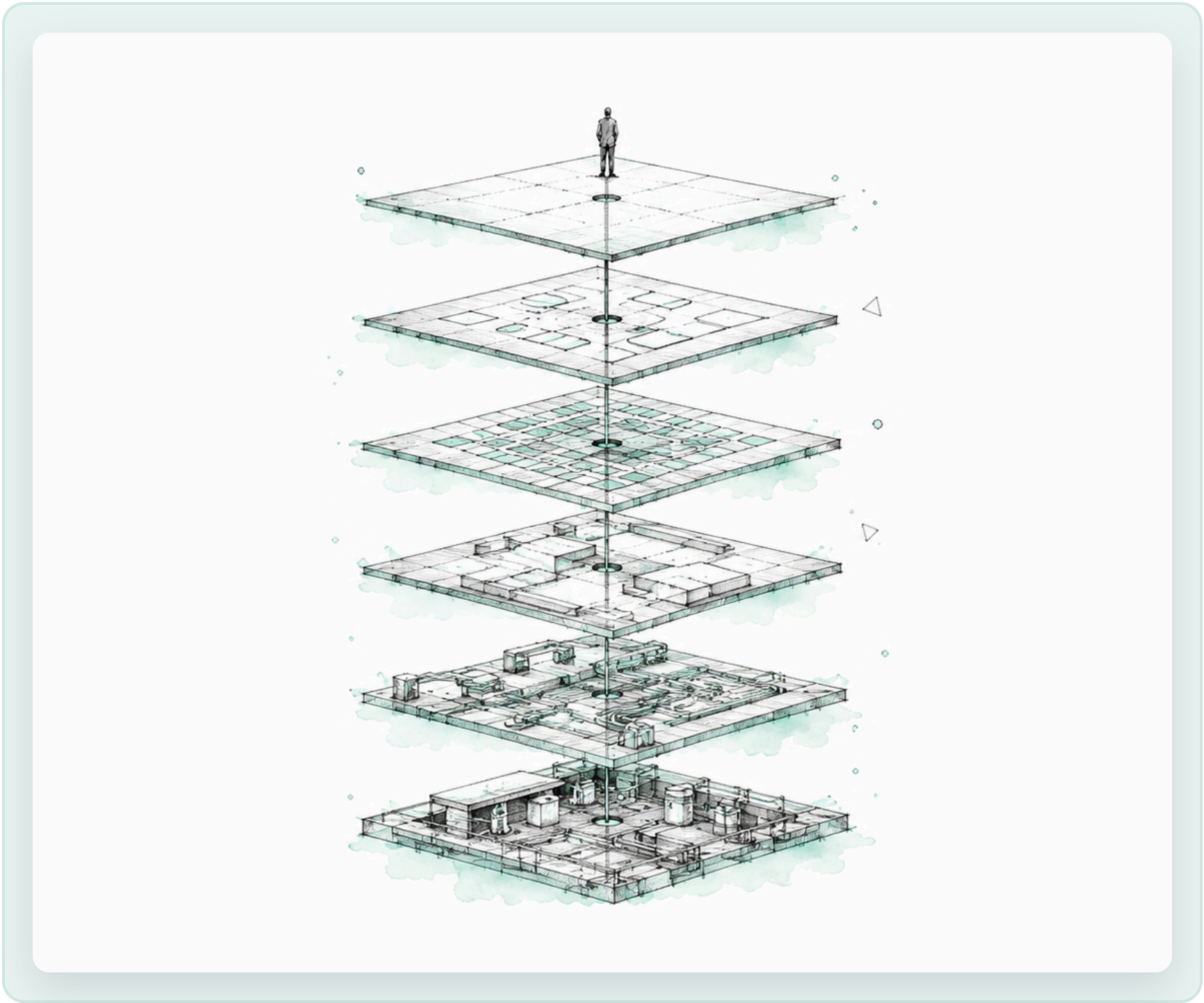
SCENE 03 • THE STACK

The Proposed AI Engineering Stack

Six layers, from human decisions down to production.



- ▲ AI sits between human intent and engineering execution.
- ▲ It doesn't replace the entire architecture — it operates within it.



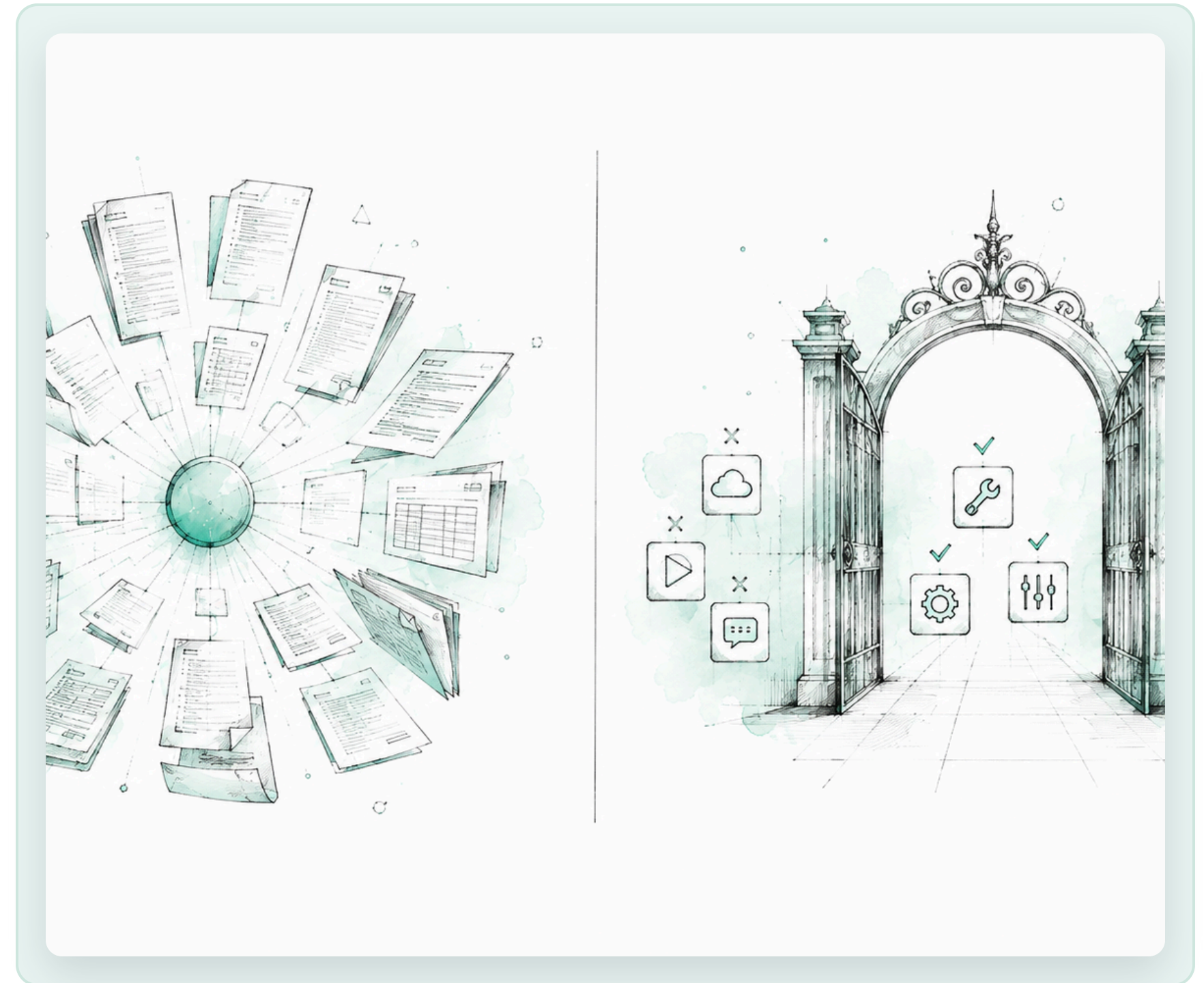
SCENE 04 • CONTEXT & TOOLS

Context Engineering & MCP

RAG feeds the agent context. MCP exposes controlled tools instead of unrestricted access.

```
get_repository()    create_branch()
run_tests()         analyze_logs()
get_metrics()       create_pull_request()
deploy_staging()    rollback()
```

- ▲ Files like AGENTS.md become part of the operational context given to AI agents.
- ▲ The agent becomes capable of acting, but the platform still controls what it is allowed to do.



SCENE 05 • SPECIALIZED AGENTS

Specialized Agents, Not One Giant AI

This is much closer to an engineering organization than to a chatbot.

ORCHESTRATOR



Architecture • Coding • Data Agents

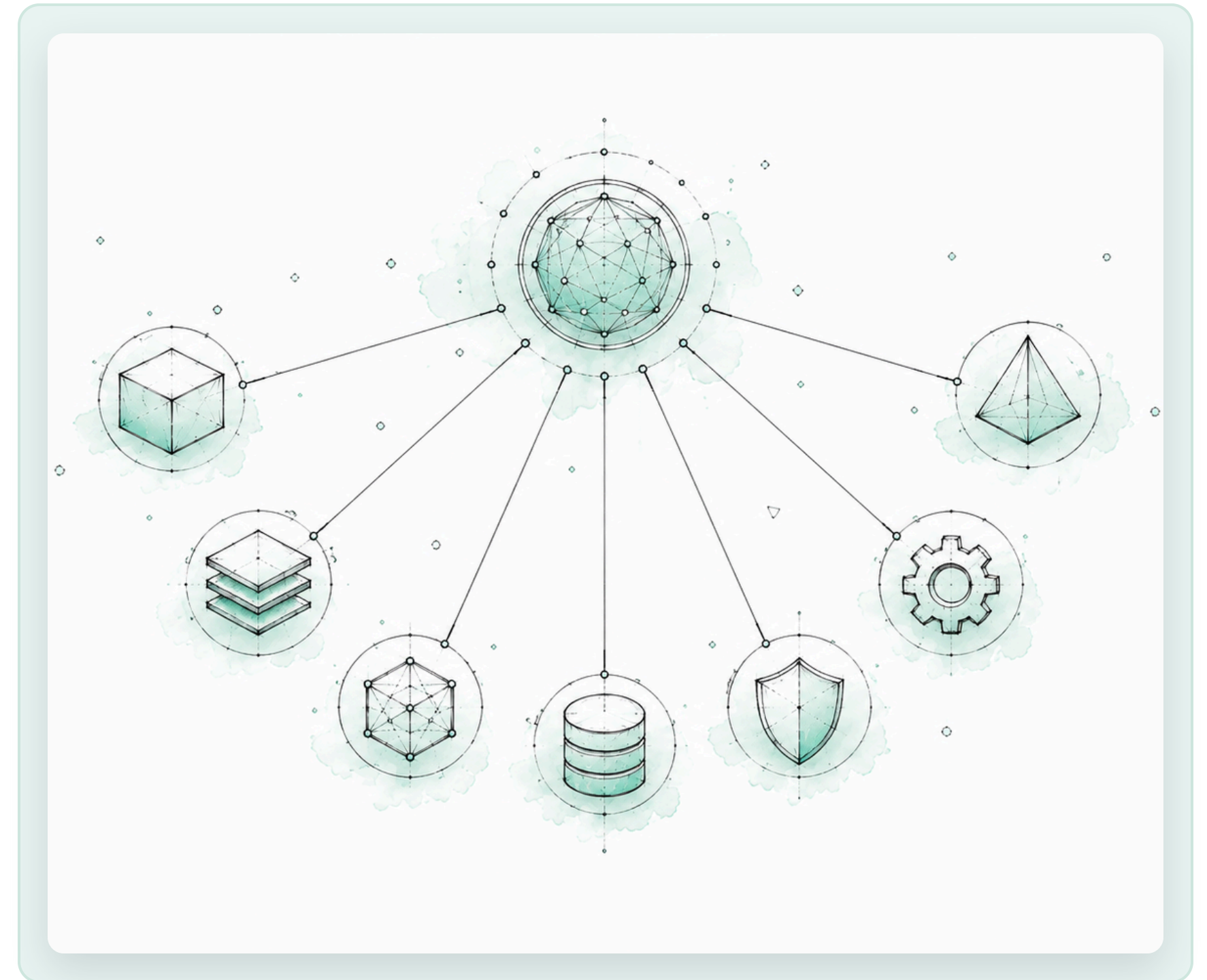


Security • Testing • ML Engineering Agents



Performance Agent → Review Agent

- ▲ Each agent has a responsibility: architecture, implementation, testing, security, performance, data, and review.
- ▲ I would not create one giant "Netflix AI Engineer" — I would create specialized agents.



SCENE 06 • SAFETY LAYER

GitHub, CI/CD & the Platform as Safety Layer

The platform evaluates and authorizes before any infrastructure action executes.

```
AI Agent → Git branch → Implementation  
→ Automated tests → Pull Request  
→ AI review → Security analysis  
→ Human review → Merge
```

- ▲ GitHub remains the collaboration layer — AI simply becomes another participant in the workflow.
- ▲ Agents should interact with capabilities, not necessarily with unrestricted infrastructure.



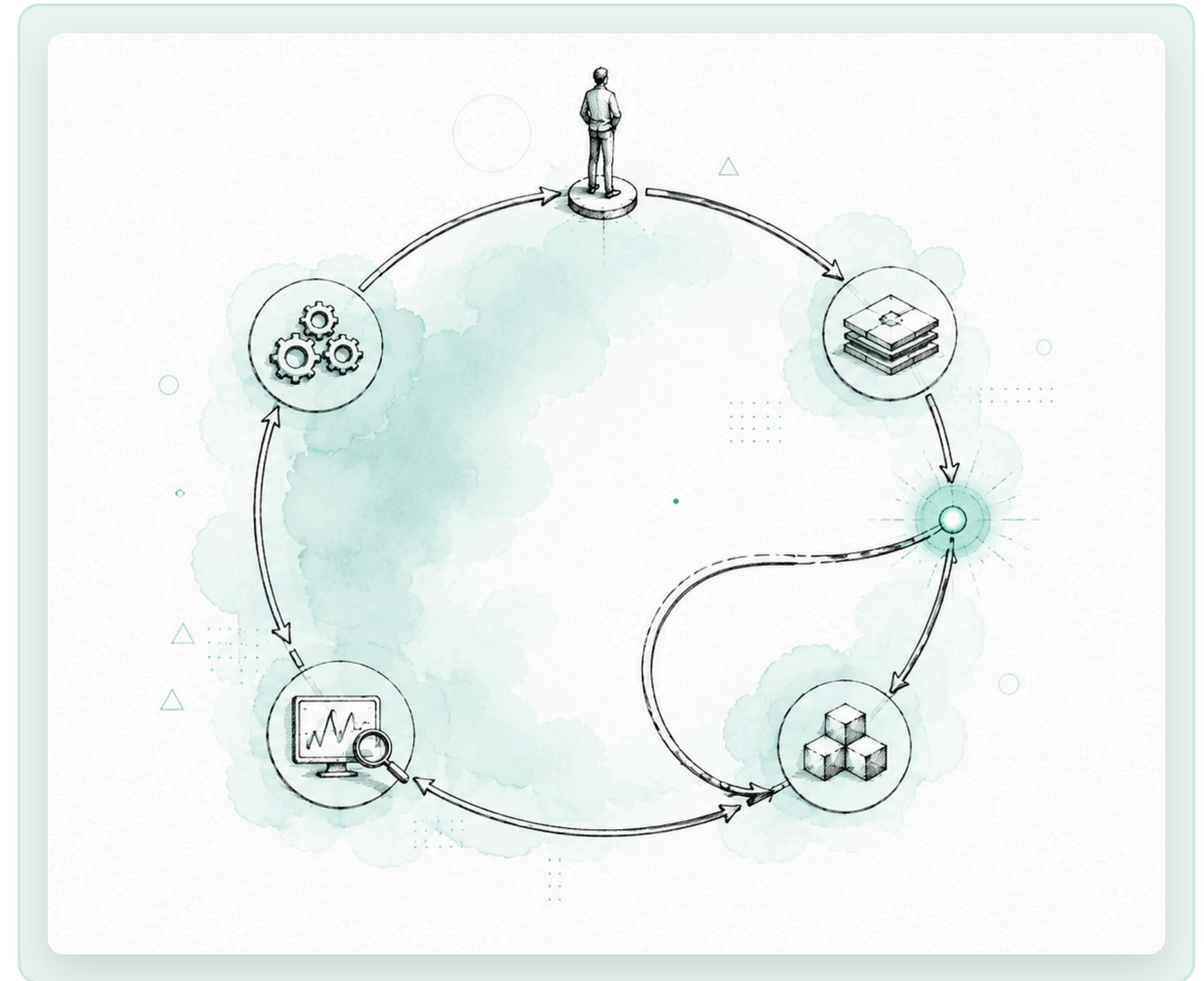
SCENE 07 • CONTROL PLANE

The Control Plane & Incident Response

Human control scales with risk.

"The higher the potential impact, the stronger the human control."

- ▲ For low risk operations, some actions could become automated; for high risk operations, human approval remains mandatory.
- ▲ AI becomes a kind of engineering control plane, creating a continuous loop between production, observability, and human feedback.



SCENE 08 • CONCLUSION

Using AI to Code vs. Engineering With AI

The more autonomous the agents become, the more important engineering expertise becomes.

"Give Netflix's engineers an AI Native engineering system capable of building, operating, and continuously improving Netflix at a scale that was previously impossible."

- ▲ The goal isn't "let AI build Netflix" — it's giving engineers an AI Native system to build, operate, and improve at scale.
- ▲ That is the difference between using AI to code and engineering with AI.

